

InsureAI

Capstone Project — Independent Learner Guide

A step-by-step self-paced roadmap covering ML, Deep Learning (ANN, CNN, LSTM) and Generative AI, integrated into a single Streamlit application.

GUVI / HCL Training Program

How to Use This Guide

This guide breaks the InsureAI capstone into 9 sequential steps, exactly following the official project document. Each step lists: what to do, what you need to know first, a checklist to confirm you're done, common mistakes to avoid, and a self-check question. Work through the steps in order — each module builds on the previous one's data understanding and saved models.

This guide intentionally does NOT give you the code. It tells you what to build, why, and how to check your own work, so you can complete the project independently. If you get stuck after trying, bring the specific error or question (not "it doesn't work") to the doubt clarification session.

 *Rule of thumb: don't move to the next step until every box in the previous step's checklist is ticked and you can explain your result in one sentence.*

Before You Start: Environment Setup

Setup Checklist

- ☐ Python 3.9+ installed; a project folder created (e.g., InsureAI/).
- ☐ Virtual environment created and activated (venv or conda).
- ☐ Git repository initialised (git init) and connected to a GitHub remote repo.
- ☐ requirements.txt started (you will keep adding to it as you install packages).
- ☐ .gitignore created with at least: .env, venv/, __pycache__/, *.pkl, *.h5, *.keras (large model files can be added via Git LFS or noted in README instead of committing if too large).
- ☐ Folder structure planned, for example:
 - data/ → raw and processed datasets
 - notebooks/ → one notebook per module (EDA + training)
 - models/ → saved .pkl / .h5 / .keras files
 - app/ → Streamlit application files
 - reports/ → prompt engineering report, screenshots, final PPT
- ☐ Kaggle account created (to download datasets).
- ☐ A free LLM API key obtained (Groq, Gemini, or OpenAI) — needed only from Step 6 onward, but good to get early.

 *Commit this empty structure to GitHub right now, before writing any code. Your first commit message can simply be 'Initial project structure'.*

STEP 1: Data Collection & Understanding (Day 1)

Objective

Get all four datasets downloaded and understand their shape before writing a single line of modeling code.

What To Do

- Search Kaggle and download: Medical Insurance Cost dataset (insurance.csv), Auto Insurance Claims Fraud dataset (insurance_claims.csv), Car Damage Detection image dataset (damaged/whole folders), and a customer review/sentiment dataset.
- For each dataset, open it in a notebook and note: number of rows/columns, data types of each column, the target variable, and any obviously missing or odd values (e.g., "?" placeholders in the fraud dataset).
- Write a short markdown cell in each notebook summarising the dataset in 3–4 sentences — this becomes the seed of your README's 'Datasets' section.

Key Concepts to Refresh

- Difference between numerical, categorical, and target columns.
- How to spot class imbalance (`value_counts()` on the target).
- Why image and text data need different handling from tabular data.

Checklist

- ☐ All 4 datasets downloaded and loaded successfully in a notebook.
- ☐ Row/column counts and dtypes documented for each dataset.
- ☐ Target variable identified for each of the 3 tabular/text tasks.
- ☐ Class imbalance in the fraud dataset noted (what % is fraud vs not?).
- ☐ Missing/placeholder values identified in the fraud dataset.

Common Mistakes

- Downloading the wrong Kaggle dataset variant (check column names match what the project expects: age, sex, bmi, children, smoker, region, charges).
- Skipping this step and jumping into modeling — you'll end up redoing preprocessing decisions later.

 *Self-check: Can you say, without looking, how many rows each dataset has and what its target column is? If not, spend 20 more minutes here before moving on.*

STEP 2: Module 1 — Machine Learning (Tabular Data) (Days 2–3)

Part A: Regression — Premium Prediction

- Perform EDA on the Medical Insurance Cost dataset: distributions of age/bmi/charges, correlation of charges with smoker status, boxplots of charges by region.
- Encode categorical columns (sex, smoker, region) — decide Label Encoding vs One-Hot and justify your choice in a comment.
- Scale numeric features (StandardScaler or MinMaxScaler) — fit the scaler on train data only.
- Split into train/test before any scaling/encoding is fitted, to avoid data leakage.
- Train at least 3 models: Linear Regression, Random Forest Regressor, XGBoost/Gradient Boosting.
- Compare using R^2 , MAE, RMSE in a small results table; select and justify the best model.

Part B: Classification — Fraud Detection

- Drop ID/leakage columns (anything that would not be known at claim-filing time, e.g. claim outcome fields).
- Handle "?" and other missing-value placeholders.
- Encode categorical columns.
- Handle class imbalance using SMOTE (on training data only, never on test data) or `class_weight='balanced'`.
- Train Logistic Regression, Random Forest, and XGBoost classifiers.
- Compare using Accuracy, Precision, Recall, F1-score, and a Confusion Matrix — for fraud detection, Recall/F1 on the minority (fraud) class matters more than raw accuracy.

Save Your Work

- Save the best regression model and the best classification model with pickle or joblib into `models/`.
- Save the fitted scaler/encoder objects too — the Streamlit app will need them to transform new user input the same way.

Checklist

- ☐ EDA plots and summary completed for the regression dataset.
- ☐ At least 3 regression models trained and compared (R^2 , MAE, RMSE).
- ☐ Best regression model achieves $R^2 \geq 0.75$ on the test set.
- ☐ Class imbalance handled correctly (fit SMOTE only on training split).
- ☐ At least 3 classification models trained and compared (Accuracy, Precision, Recall, F1, Confusion Matrix).
- ☐ Best classifier achieves $F1 \geq 0.70$ on the fraud (minority) class.
- ☐ Best models + preprocessing objects saved to models/.

Common Mistakes

- Applying SMOTE before the train/test split (this leaks synthetic copies of test data into training).
- Judging the fraud model only on accuracy — with imbalance, a model that predicts "no fraud" every time can still show high accuracy.
- Forgetting to save the scaler/encoder, then being unable to correctly transform new inputs later in the Streamlit app.

 *Self-check: If someone gave you a brand-new customer's age, sex, bmi, children, smoker, region right now, could you walk through, step by step, exactly what your saved pipeline would do to produce a premium prediction?*

STEP 3: Module 2 — Deep Learning (ANN) (Day 3–4)

What To Do

- Build an ANN (Keras/TensorFlow) with 2–3 hidden layers, for premium prediction (regression) and/or fraud detection (classification) — using the same preprocessed tabular data from Step 2.
- Use ReLU activation in hidden layers; linear output activation for regression, sigmoid output activation for binary classification.
- Use the Adam optimizer, an appropriate loss function (MSE for regression, binary cross-entropy for classification), EarlyStopping to prevent overfitting, and Dropout layers for regularisation.
- Plot training vs validation loss curves across epochs.
- Build a small comparison table: ANN performance vs your best ML model from Module 1, on the same metrics.

Checklist

- ☐ ANN architecture defined with 2–3 hidden layers, correct activations for the task.
- ☐ EarlyStopping and Dropout used and explained (why these layers/callbacks, not just that they exist).
- ☐ Training vs validation loss curve plotted and visually checked for overfitting.
- ☐ Comparison table: ANN vs best classical ML model, same metrics, same test set.

Common Mistakes

- Comparing ANN and ML model results on different test splits — always evaluate on the identical held-out test set.
- Ignoring a widening gap between training loss and validation loss (a sign of overfitting) instead of addressing it with Dropout/EarlyStopping/more data.

 *Self-check: Looking at your loss curve, can you point to the epoch where validation loss stopped improving, and explain what EarlyStopping did about it?*

STEP 4: Module 3 — CNN (Vehicle Damage Image Classification) (Days 4–5)

What To Do

- Load the Car Damage Detection dataset (damaged / whole folders).

- Preprocess: resize images (e.g., 224×224), normalise pixel values, and apply data augmentation (rotation, flip, zoom) to the training set only.
- Approach 1: Build a simple CNN from scratch — Conv2D → MaxPooling → Flatten → Dense layers.
- Approach 2 (recommended): Use Transfer Learning with a pre-trained backbone (MobileNetV2 or VGG16), freezing the base layers initially.
- Compare the scratch CNN and the transfer-learning model on test accuracy and confusion matrix.
- Save the best-performing model (.h5 or .keras).

Checklist

- ☐ Images resized/normalised consistently for train, validation, and test sets.
- ☐ Augmentation applied only to the training data, not validation/test.
- ☐ Both a scratch CNN and a transfer-learning model trained.
- ☐ Test accuracy $\geq 85\%$ achieved by the better model.
- ☐ Confusion matrix and a handful of sample predictions (correct and incorrect) reviewed and shown in the notebook.
- ☐ Best model saved (.h5/.keras).

Common Mistakes

- Augmenting the validation/test set (this distorts your true performance measurement).
- Not freezing the pre-trained base layers initially when doing transfer learning, which can destroy the useful pre-trained weights early in training.
- Using too few epochs to judge the scratch CNN fairly against the pre-trained model.

 *Self-check: Pick one misclassified image from your confusion matrix review — can you guess, just by looking at it, why the model got it wrong (lighting, angle, ambiguous damage)?*

STEP 5: Module 4 — RNN / LSTM (Customer Review Sentiment) (Days 6–7)

What To Do

- Use an insurance/customer review dataset with text and a rating or sentiment label (if unavailable, use any labelled review dataset and relabel: rating ≥ 4 = Positive, ≤ 2 = Negative).
- Preprocess text: lowercase, remove punctuation/stopwords, tokenise with the Keras Tokenizer (vocab ~10,000 words), pad sequences to a fixed length (e.g., 100).
- Build a model: Embedding layer → LSTM layer → Dense output layer for binary sentiment classification.
- Evaluate using accuracy and F1-score on the test set.
- Test the trained model on a few of your own example sentences, e.g. "My claim was settled very fast", and confirm the predicted sentiment matches your expectation.

Checklist

- ☐ Text cleaned and tokenised; sequences padded to a consistent length.
- ☐ Embedding → LSTM → Dense architecture built and trained.
- ☐ Test accuracy $\geq 85\%$ achieved.
- ☐ At least 3 custom example sentences tested and results sanity-checked.
- ☐ Model (and the fitted tokenizer!) saved for reuse in the Streamlit app.

Common Mistakes

- Fitting the tokenizer on the full dataset instead of just the training split (a mild form of leakage).

- Forgetting to save the tokenizer alongside the model — without it, the Streamlit app cannot correctly convert new user text into the same numeric sequences.
- Using too short a max sequence length, truncating meaningful context from longer reviews.

 *Self-check: If you feed the model a clearly sarcastic or mixed review, does the prediction still make sense? It's fine if it doesn't — but you should be able to explain why in your documentation.*

STEP 6: Module 5 — Generative AI, Prompt Engineering & LLM API (Days 8–9)

What To Do

- Get a free/low-cost API key (Groq, Google Gemini, or OpenAI). Store it in a .env file and load it with python-dotenv — never hard-code it, and confirm .env is in .gitignore before your first commit that touches this module.
- Create a small FAQ text of 10–15 insurance Q&A pairs (policy types, claim process, documents needed) to ground your chatbot's answers.
- Build three features by calling the LLM API:
 - Policy Q&A Chatbot — a system prompt that restricts the bot to insurance topics only, using your FAQ text as context.
 - Claim Summariser — takes a long claim description and returns a short structured summary (incident, date, damage, estimated severity).
 - Email Drafter — generates a professional claim-status email to a customer from given claim details.
- Demonstrate and document at least 4 prompt engineering techniques: Zero-shot, Few-shot (with examples), Role/Persona prompting ("You are a senior insurance claims officer..."), Chain-of-thought/step-by-step instructions, and structured-output prompting (asking the model to return JSON).
- Write a short before/after comparison: show one weak prompt's output next to the same task with an engineered prompt, and explain what changed and why the second is better.

Checklist

- ☐ API key loaded from .env, never committed to Git.
- ☐ FAQ knowledge text created and used as grounding context.
- ☐ All 3 GenAI features (chatbot, summariser, email drafter) working via live API calls.
- ☐ At least 4 distinct prompting techniques demonstrated and documented with screenshots.
- ☐ Weak-prompt vs engineered-prompt comparison written up.
- ☐ Basic error handling in place for API failures (timeouts, rate limits, empty responses).

Common Mistakes

- Hard-coding the API key directly in a notebook cell (even temporarily) — this is easy to accidentally commit.
- Writing a system prompt so loose that the chatbot answers unrelated general-knowledge questions instead of staying on insurance topics.
- Asking for JSON output but not validating/parsing it defensively — LLMs occasionally add stray text around the JSON.

 *Self-check: Ask your chatbot something completely off-topic (e.g., "What's the capital of France?"). Does your system prompt correctly redirect it back to insurance topics?*

STEP 7: Integration & Deployment — The Streamlit App (Days 9–10)

What To Do

- Build one Streamlit app with a sidebar menu: Home, Premium Prediction, Fraud Detection, Damage Detection (image upload), Review Sentiment, InsureAI Chatbot.
- Load your saved models (and scalers/encoders/tokenizer) once, and connect each sidebar page to the correct model/API call.
- Keep the app modular — one function per feature/page, not one giant script.
- Handle errors gracefully: invalid numeric inputs, wrong image formats, missing image uploads, API timeouts or failures.
- (Optional bonus) Deploy the app on Streamlit Community Cloud or Hugging Face Spaces.

Checklist

- ☐ All 5 modules accessible from one running Streamlit app.
- ☐ Each page correctly loads its model and preprocessing objects.
- ☐ Invalid input handled without crashing the app (e.g., text where a number was expected, non-image files).
- ☐ API failures in the chatbot page show a friendly message, not a raw traceback.
- ☐ Code organised into functions with comments; PEP-8 followed.

Common Mistakes

- Reloading large models on every user interaction instead of caching them (use Streamlit's caching so the app stays responsive).
- Forgetting that the scaler/encoder/tokenizer must be loaded and applied identically to how they were used in training.

 *Self-check: Deliberately try to break your own app — upload a PDF where an image is expected, type letters into a number field, disconnect the internet before a chatbot call. Does it fail gracefully?*

STEP 8: Documentation & Deliverables (Day 10)

What To Prepare

- Complete source code: one Jupyter notebook per module (EDA + training) plus the Streamlit app's Python scripts.
- Saved model files: .pkl for ML models, .h5/.keras for DL/CNN/LSTM models.
- Prompt Engineering Report (1–2 pages): the exact prompts used, techniques applied, and before/after output comparison screenshots.
- GitHub repository with a clear README: project overview, setup steps, screenshots, and a results table for all 5 modules.
- Final project documentation/PPT: problem statement, approach, results across all 5 modules, and key learnings.
- A short demo video or a live demo of the Streamlit application.

Checklist

- ☐ Every notebook runs top-to-bottom without errors on a fresh restart.
- ☐ requirements.txt is complete and accurate (test by installing into a fresh venv if time allows).
- ☐ README includes setup instructions someone unfamiliar with the project could follow.
- ☐ Results table in README/PPT covers all 5 modules with their key metric.
- ☐ Prompt Engineering Report attached with screenshots.
- ☐ Demo video recorded OR live demo rehearsed.
- ☐ All commits present in GitHub history (not just one giant final commit) — commit after each module.

STEP 9: Final Review Against Evaluation Metrics *(Before Submission)*

Before you submit, walk through this table and confirm you can show evidence for each row.

Area	What's Evaluated
Regression	R ² Score, MAE, RMSE — target R ² ≥ 0.75
Classification (Fraud & Sentiment)	Accuracy, Precision, Recall, F1-Score, Confusion Matrix, ROC-AUC
Deep Learning models	Training/validation loss & accuracy curves; overfitting handling via Dropout/EarlyStopping
CNN	Test accuracy (≥ 85%), confusion matrix, sample predictions on unseen images
GenAI Module	Correctness/relevance of responses, documented prompt techniques, safe API key handling, error handling
Application	Working Streamlit app, code quality (modular, commented), GitHub repo with README, clarity of final demo

 If any row above feels shaky, that's exactly what to revisit before your live evaluation — not something to hope nobody asks about.

Getting Help Without Losing Independence

Try to solve the problem yourself first: re-read the relevant step above, check your error message carefully, and search for the exact error text. If you're still stuck after a genuine attempt, use the sessions below — but come with a specific question ("my SMOTE step throws X error on this line"), not a general "it's not working".

Session	Purpose	Timing
Project Doubt Clarification (DS/AIML)	Project & class topic doubts	Mon–Fri 3:00–4:00 PM; Sat 2:00–3:00 PM
Project Doubt Clarification (BMAI/BADM)	Project & class topic doubts	Tue, Thu, Sat 3:00–4:00 PM
Live Evaluation Session (Capstone/Final Project)	Showcase project, get real-time feedback	Mon–Sat 11:00 AM–12:00 PM; Mon–Fri 5:00–6:00 PM (booking form opens Sat after 2 PM & Sunday)

Live Evaluation booking link: <https://forms.gle/1m2Gsro41fLtZurRA>

Suggested Timeline at a Glance

Day(s)	Focus
Setup	Environment, repo, folder structure
Day 1	Step 1 — Data collection & understanding
Days 2–3	Step 2 — Module 1: ML regression + classification
Days 3–4	Step 3 — Module 2: ANN
Days 4–5	Step 4 — Module 3: CNN / transfer learning
Days 6–7	Step 5 — Module 4: LSTM sentiment
Days 8–9	Step 6 — Module 5: GenAI + prompt engineering
Days 9–10	Step 7 — Streamlit integration & deployment
Day 10	Step 8–9 — Documentation, deliverables, final review

 This maps the official Week 1 / Week 2 timeline into finer daily targets — adjust to your own pace, but don't let any single module eat both weeks.